MCMP117 · Week 5 · Course Notes

Fundamentals of Operating Systems

CPU Scheduling Algorithms & Process Synchronisation

Who runs next, and how do threads share data without breaking it?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Draw the Gantt chart of a job set under FCFS, SJF, SRTF, priority and Round Robin scheduling, and compute the waiting, turnaround and response time of every process.
2. Explain the convoy effect, why SJF minimises the average waiting time, and how the next CPU burst is predicted by exponential averaging.
3. Explain starvation and aging, and the trade-off in choosing the Round Robin time quantum.
4. Describe, at overview level, multilevel feedback queues and the fair scheduler used by Linux.
5. Explain a race condition using the load–add–store sequence of `counter++`; state the critical-section problem and its three requirements, and describe Peterson's solution, atomic instructions, mutex locks and semaphores.
6. Solve the bounded-buffer problem with semaphores and recognise a simple deadlock.
7. Read, compile and run small `pthreads` programs that show a race and its fix.

Contents

1 Recap and the Running Example 1

2 First-Come, First-Served (FCFS) 2

3 Shortest-Job-First Scheduling 3

4 Priority Scheduling 5

5 Round Robin Scheduling 6

6 Multilevel Queues and Real Schedulers (Overview) 8

7 Race Conditions 10

8 The Critical-Section Problem 12

9 Peterson's Solution 13

10 Hardware Support: Atomic Instructions 13

11 Mutex Locks 14

12 Semaphores 14

13 The Bounded-Buffer Problem 15

14 A Warning: Deadlock 16

15 Hands-on: Threads, Races and Locks in C 16

16 Summary 19

17 Glossary 19

18 Try It Yourself 20

19 Review Questions 20

20 Looking Ahead: Week 6 21

1 Recap and the Running Example

Last week we met the **CPU scheduler**, which chooses one process from the ready queue, and the **dispatcher**, which actually gives it the CPU (context switch, switch to user mode, jump to the right instruction). Scheduling can be **non-preemptive** (a process keeps the CPU until it terminates or blocks for I/O) or **preemptive** (the OS may take the CPU away, typically on a timer interrupt or when a more important process becomes ready). We judge a schedule by the following criteria, where every time is measured for one process:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}},$$
$$T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}},$$
$$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}.$$

The waiting time is the total time a process spends *in the ready queue*. For a non-preemptive algorithm a process runs in one piece, so its waiting time equals its response time. CPU utilisation and throughput should be as high as possible; turnaround, waiting and response time as low as possible.

1.1 One job set for every algorithm

To compare algorithms fairly we use the *same* four processes throughout Part 1. All times are in milliseconds (ms).

Process	Arrival time	CPU burst	Priority
P1	0	7	3
P2	2	4	1
P3	4	1	4
P4	5	4	2

Conventions (state them in every exam answer, because textbooks differ):

- Context-switch time is taken as 0; we count the number of switches separately.
- Ties are broken by arrival time, then by process name.
- A *smaller* priority number means a *higher* priority.
- In Round Robin, a process that arrives at the same instant as a quantum expires enters the ready queue *before* the preempted process.

The total CPU work is $7 + 4 + 1 + 4 = 16$ ms and the CPU is never idle, so every schedule below ends at $t = 16$. A **Gantt chart** is a time line divided into boxes; each box shows which process uses the CPU during that interval.

Every number in these notes and on the slides was computed by a short Python program in the `code/` folder, `sched_check.py`. Run `python3 sched_check.py` to print all Gantt charts and tables, or edit its list `JOBS` to try your own job set.

2 First-Come, First-Served (FCFS)

The simplest policy: the ready queue is a FIFO (first-in, first-out) queue, and the process that arrived first gets the CPU and keeps it until it finishes or blocks. FCFS is non-preemptive.

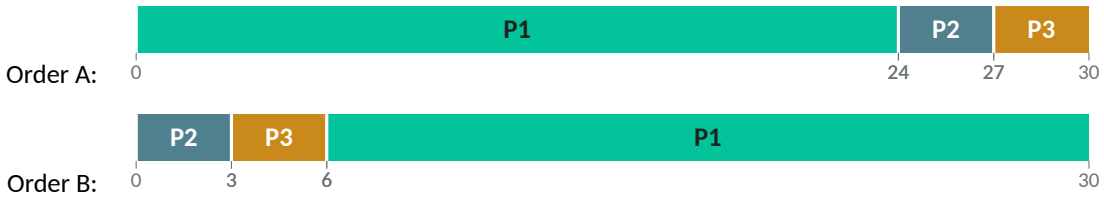


Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	7	7	0	0
P2	2	4	11	9	5	5
P3	4	1	12	8	7	7
P4	5	4	16	11	7	7
Average				8.75	4.75	4.75

For example, P3 arrives at $t = 4$, starts at $t = 11$ and finishes at $t = 12$: $T_{\text{turnaround}} = 12 - 4 = 8$ ms and $T_{\text{waiting}} = 8 - 1 = 7$ ms. The average waiting time is $(0 + 5 + 7 + 7)/4 = 4.75$ ms.

2.1 The convoy effect

FCFS is fair in the "queue at the bank" sense, but the average waiting time depends heavily on the order in which processes happen to arrive. Take three jobs that all arrive at $t = 0$ with bursts of 24, 3 and 3 ms.



$$\text{Order A:}\quad \overline{T}_{\text{waiting}} = \frac{0 + 24 + 27}{3} = \frac{51}{3} = 17 \text{ ms},$$

$$\text{Order B:}\quad \overline{T}_{\text{waiting}} = \frac{0 + 3 + 6}{3} = \frac{9}{3} = 3 \text{ ms}.$$

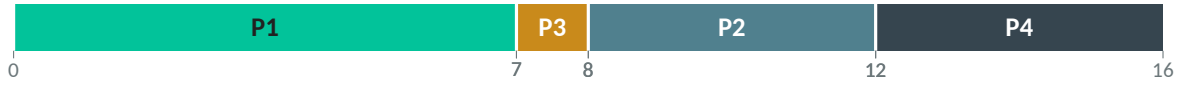
Same jobs, same algorithm, almost six times less waiting. When many short, I/O-bound processes are stuck behind one long CPU-bound process, the whole system slows down, like cars behind a slow truck on a mountain road. This is the **convoy effect**: the I/O devices sit idle while the short jobs wait, and then the CPU sits idle while they all do I/O.

3 Shortest-Job-First Scheduling

Order B above suggests a better rule: *run the shortest job first*. Strictly, the rule looks at the length of the *next CPU burst*, not of the whole job, so a more accurate name would be "shortest-next-CPU-burst".

3.1 Non-preemptive SJF

Whenever the CPU becomes free, choose the ready process with the shortest next burst; once started, it runs to completion.



At $t = 0$ only P1 is ready, so it runs to $t = 7$. At $t = 7$ the ready queue holds P2 (4 ms), P3 (1 ms) and P4 (4 ms); P3 is shortest. At $t = 8$, P2 and P4 tie at 4 ms and P2 arrived first.

The waiting times are P1 0, P2 $12 - 2 - 4 = 6$, P3 $8 - 4 - 1 = 3$ and P4 $16 - 5 - 4 = 7$ ms; the turnaround times are 7, 10, 4 and 11 ms.

The average waiting time falls from 4.75 ms (FCFS) to 4 ms. Note the remaining weakness: P1 started at $t = 0$ simply because nobody else was there, and then the very short P3 had to wait for it.

3.2 Preemptive SJF: shortest-remaining-time-first (SRTF)

In the preemptive version, every time a new process arrives the scheduler compares its burst with the *remaining* time of the running process; if the newcomer is shorter, it takes the CPU.



- $t = 2$: P2 arrives needing 4 ms; P1 still needs $7 - 2 = 5$ ms. Since $4 < 5$, P1 is preempted.
- $t = 4$: P3 arrives needing 1 ms; P2 still needs 2 ms. P2 is preempted.
- $t = 5$: P3 is done. Remaining times: P2 2, P4 4, P1 5. So P2 runs 5–7, then P4 7–11, then P1 11–16.

Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	16	16	9	0
P2	2	4	7	5	1	0
P3	4	1	5	1	0	0
P4	5	4	11	6	2	2
Average				7	3	0.5

With preemption a process may run in several pieces, so we compute the waiting time from the formula rather than by adding up gaps: P1 finishes at 16, so $T_{\text{waiting}}(\text{P1}) = (16 - 0) - 7 = 9$ ms (check: P1 waited from 2 to 11, i.e. 9 ms). The average waiting time is $(9 + 1 + 0 + 2)/4 = 3$ ms, the best value of the day; the long job P1 pays for the speed of the short ones.

3.3 Why shortest-first is optimal

Optimality of SJF

If all jobs are available at the same time, running them in order of increasing burst length gives the minimum possible average waiting time. When jobs arrive at different times, SRTF gives the minimum (if context switches are free).

The proof is an *exchange argument*. Consider any schedule in which a longer job L runs immediately before a shorter job S ($L > S$), starting at some time t_0. Swap the two:

- Every job before the pair starts at the same time as before; every job after the pair also starts at the same time, because $L + S$ is unchanged.
- Inside the pair, the waiting times were t_0 (for L) and $t_0 + L$ (for S); after the swap they are t_0 and $t_0 + S$.

The total waiting time therefore decreases by $L - S > 0$. Any schedule with a longer job before a shorter one can be improved, so the shortest-first order is optimal. In the convoy example, order B is exactly the shortest-first order.

3.4 The catch: predicting the next CPU burst

SJF needs the length of the next CPU burst *before* the process runs. For long-term batch scheduling, users could be asked to estimate the run time of their job; for CPU scheduling there is no way to know. The OS can only *predict*, assuming that the next burst will be similar to recent ones. The standard predictor is the **exponential average**

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \le \alpha \le 1,$$

where t_n is the measured length of the n-th burst and τ_n is the value that had been predicted for it. Expanding the recursion shows where the name comes from:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + (1-\alpha)^2 \alpha t_{n-2} + \cdots + (1-\alpha)^{n+1}\tau_0.$$

Since $1-\alpha < 1$, older bursts get exponentially smaller weights. With $\alpha = 0$ the prediction never changes (recent history is ignored); with $\alpha = 1$ only the last burst counts. A common choice is $\alpha = 1/2$.

Exponential averaging, $\alpha = 1/2$, $\tau_0 = 10$ ms

The process has actual bursts $t_0, t_1, \cdots = 6, 4, 6, 4, 13, 13, 13$ ms. The first two steps:

$$\tau_1 = \tfrac{1}{2}\cdot 6 + \tfrac{1}{2}\cdot 10 = 8 \text{ ms},$$
$$\tau_2 = \tfrac{1}{2}\cdot 4 + \tfrac{1}{2}\cdot 8 = 6 \text{ ms}.$$

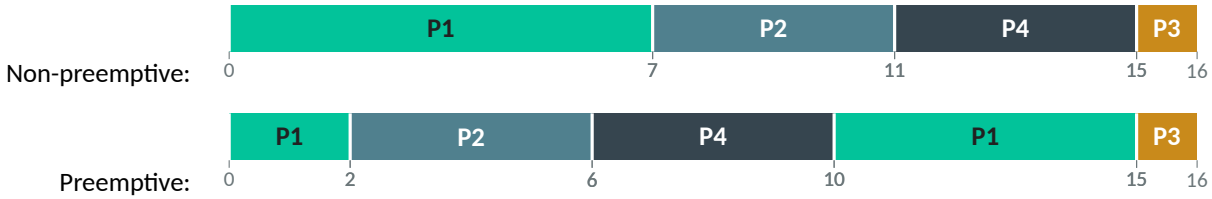
Continuing in the same way:

n	0	1	2	3	4	5	6	7
actual burst t_n (ms)	6	4	6	4	13	13	13	?
prediction τ_n (ms)	10	8	6	6	5	9	11	12

When the behaviour of the process changes (bursts jump to 13 ms), the prediction follows with a delay: $5 \to 9 \to 11 \to 12$. This lag is the price of smoothing out random fluctuations.

4 Priority Scheduling

Each process is given a **priority**, and the CPU goes to the ready process with the highest priority. In this course (as in Linux and in Silberschatz) a smaller number means a higher priority. SJF is the special case in which the priority is the predicted burst length. Priority scheduling can be non-preemptive or preemptive.



In the preemptive version P2 (priority 1) takes the CPU from P1 (priority 3) as soon as it arrives at $t = 2$ and runs to 6; then P4 (priority 2, arrived at 5) runs 6–10; P1 finishes 10–15; P3 runs last.

		Non-preemptive		Preemptive	
Process	**Priority**	**Waiting**	**Response**	**Waiting**	**Response**
P1	3	0	0	8	0
P2	1	5	5	0	0
P3	4	11	11	11	11
P4	2	6	6	1	1
Average		5.5	5.5	5	3

The average turnaround times are 9.5 ms (non-preemptive) and 9 ms (preemptive). Priority scheduling does not try to minimise averages: it serves *importance*. Priorities can be *internal* (computed by the OS from measurable quantities such as memory needs or the ratio of I/O to CPU bursts) or *external* (set from outside: the importance of the job, the department that pays for it).

4.1 Starvation and aging

P3, with the lowest priority, waited 11 ms in both versions. In a busy system where higher-priority processes keep arriving, a low-priority process may *never* run. This is called **starvation** or *indefinite blocking*.

> **A job that waited six years**
>
> According to a widely repeated story, when the IBM 7094 at MIT was shut down in 1973, the operators found a low-priority process that had been submitted in 1967 and had never been run. Whether or not every detail is true, it illustrates the problem well.

The standard cure is **aging**: gradually increase the priority of a process the longer it waits. For example, if priorities range from 127 (lowest) to 0 (highest) and a waiting process gains one priority level every second, even a process that starts at 127 reaches priority 0 after at most

$$127 \times 1 \text{ s} = 127 \text{ s} = 2 \text{ min } 7 \text{ s}.$$

Aging turns “maybe never” into “within a bounded time”. We will meet the same idea again as the *bounded-waiting* requirement in Section 8.

5 Round Robin Scheduling

Round Robin (RR) is FCFS with preemption, designed for time-sharing systems. A small unit of time, the **time quantum** (or time slice) q, is defined, typically 10–100 ms. The ready queue is treated as a circular FIFO queue:

1. The scheduler takes the process at the head of the queue and sets a timer to interrupt after q.
2. If the CPU burst ends before q (the process finishes or blocks for I/O), the process leaves the CPU voluntarily and the next one starts at once.
3. If the timer goes off first, the process is preempted and put at the *tail* of the ready queue.

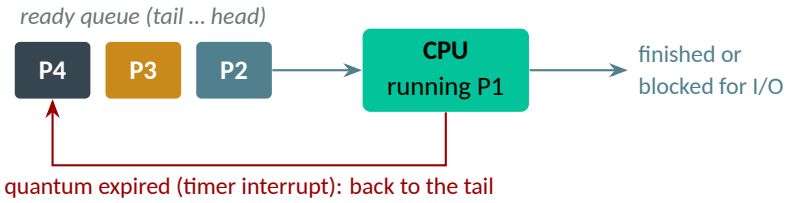
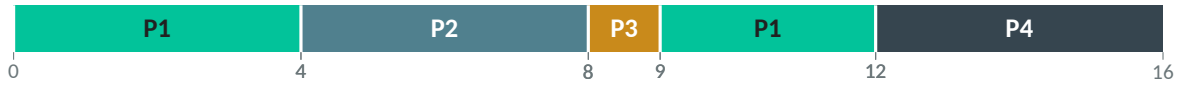


Figure 1: Round Robin.

With n processes in the ready queue, each gets $1/n$ of the CPU time in chunks of at most q, and no process waits more than $(n-1)\,q$ before its next turn. This bound is what makes RR good for response time.

5.1 Worked example: $q = 4$ ms



Follow the ready queue (head on the left):

- $t = 0$: only P1; it runs for a full quantum, 0–4.
- $t = 4$: P2 (arrived at 2) and P3 (arrived at 4) are queued; P1 joins at the tail (our convention: P3, arriving exactly at 4, goes before P1). Queue: P2, P3, P1.
- $t = 8$: P2 has used its 4 ms and is finished. P4 arrived at 5. Queue: P3, P1, P4.
- $t = 9$: P3 finished after 1 ms. P1 needs 3 more ms and finishes at 12; then P4 runs 12–16.

Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	7	12	12	5	0
P2	2	4	8	6	2	2
P3	4	1	9	5	4	4
P4	5	4	16	11	7	7
Average				8.5	4.5	3.25

5.2 Worked example: $q = 2$ ms



Time	Event	Ready queue afterwards (head first)
0	P1 starts	—
2	P2 arrived; P1's quantum expires	P2, P1
4	P3 arrived; P2's quantum expires	P1, P3, P2
6	P4 arrived at 5; P1's quantum expires	P3, P2, P4, P1
7	P3 finishes (1 ms burst)	P2, P4, P1
9	P2 finishes	P4, P1
11	P4's quantum expires	P1, P4
13	P1's quantum expires	P4, P1
15	P4 finishes; P1 runs its last 1 ms	—

Finishing times are P1 16, P2 9, P3 7, P4 15, so the waiting times are 9, 3, 2 and 6 ms and the response times 0, 0, 2 and 4 ms (averages: waiting 5, turnaround 9, response 1.5 ms).

Compared with $q = 4$: the average response time improves from 3.25 to 1.5 ms, but the average waiting time grows from 4.5 to 5 ms and the number of context switches doubles from 4 to 8.

5.3 Choosing the time quantum

Quantum q	Avg waiting (ms)	Avg response (ms)	Context switches
1 ms	5.5	0.75	14
2 ms	5	1.5	8
4 ms	4.5	3.25	4
8 ms	4.75	4.75	3

- q **too large:** RR degenerates into FCFS. With $q = 8$ ms, which is longer than every burst in our job set, the schedule is *identical* to FCFS.
- q **too small:** the CPU spends a large share of its time switching. If each switch costs s, then each quantum of useful work q costs one switch, and the fraction of CPU time lost is

$$\frac{s}{q + s}.$$

  With a (deliberately pessimistic) switch cost of $s = 0.1$ ms this gives 9.1% for $q = 1$ ms, 2.4% for $q = 4$ ms, 1.0% for $q = 10$ ms and 0.1% for $q = 100$ ms.

Real context switches cost a few microseconds, plus an indirect cost: the caches and the TLB are "cold" for the newly scheduled process. A common rule of thumb is to choose q so that about 80% of CPU bursts are shorter than q; interactive processes then usually finish their burst within one quantum.

6 Multilevel Queues and Real Schedulers (Overview)

6.1 Multilevel queue and multilevel feedback queue

Real systems combine several algorithms. In a **multilevel queue** scheduler the ready queue is split into several queues, for example *foreground* (interactive) processes scheduled by RR and *background* (batch) processes scheduled by FCFS, with fixed priority between the queues. A process stays in its queue permanently.

A **multilevel feedback queue** (MLFQ) lets processes move between queues according to their behaviour. A classic example has three queues:

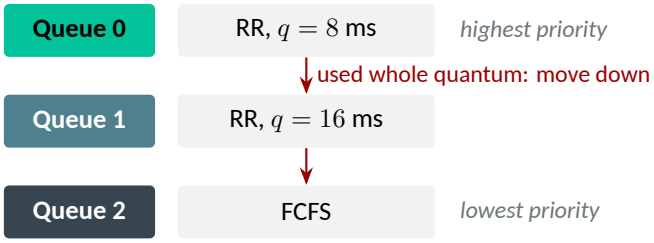


Figure 2: A three-level feedback queue.

A new process enters queue 0. If it does not finish its burst within 8 ms, it moves to queue 1; if it again uses the whole 16 ms quantum, it moves to queue 2. Queue 1 runs only when queue 0 is empty, and so on. Short interactive bursts therefore finish at high priority, while long CPU-bound processes sink to the bottom. A process that waits too long in a low queue can be moved up again: this is aging. MLFQ approximates SJF *without* knowing the burst lengths in advance, by watching how each process behaved. It is the most general scheme, and the hardest to tune.

6.2 What Linux does

Linux does not use a fixed time quantum. Its scheduler for normal processes aims at *fairness*: every runnable task should receive a share of the CPU proportional to its **weight**.

- From kernel 2.6.23 (2007) to 6.5 this was the **Completely Fair Scheduler (CFS)**. Each task has a *virtual run time*, its CPU time divided by its weight; CFS always runs the task with the smallest virtual run time.
- Since kernel 6.6 (2023) the default is **EEVDF** (*Earliest Eligible Virtual Deadline First*). It keeps the fairness goal, but additionally gives each task a virtual deadline, so that tasks that need low latency can run sooner.
- The weight comes from the **nice value**, from -20 (most CPU) to $+19$ (least CPU), default 0. Each nice step changes the weight by a factor of about 1.25; nice 0 has weight 1024 and nice 10 has weight 110.
- Real-time scheduling classes (`SCHED_FIFO`, `SCHED_RR`, priorities 1–99) always run before normal tasks.

Measuring nice on a real kernel

Two copies of an endless loop `for(;;) x++;` (program `code/cpu_hog.c`) are pinned to the same CPU, one at nice 0, one at nice 10. The predicted shares are

$$\frac{1024}{1024+110} \approx 90.3\%, \qquad \frac{110}{1024+110} \approx 9.7\%, \qquad \text{ratio } \frac{1024}{110} \approx 9.31.$$

Measured on our build machine (Linux 6.18, i.e. EEVDF), after 10 s:

```
$ taskset -c 1 ./cpu_hog &
$ taskset -c 1 nice -n 10 ./cpu_hog &
$ sleep 10; ps -o pid,ni,cls,pcpu,comm -C cpu_hog
  PID  NI CLS %CPU COMMAND
 7150   0  TS 88.0 cpu_hog
 7151  10  TS  9.4 cpu_hog
```

The measured ratio $88.0/9.4 \approx 9.4$ agrees well with the prediction. (The two percentages do not add up to 100 because `ps` reports the average since each process started, and the two did not start at exactly the same moment. `CLS` `TS` means "time sharing", the normal class.)

6.3 Comparison on our job set

Algorithm	Preemptive	Avg waiting	Avg turnaround	Avg response	Switches
FCFS	no	4.75	8.75	4.75	3
SJF	no	4	8	4	3
SRTF	yes	3	7	0.5	5
Priority	no	5.5	9.5	5.5	3
Priority	yes	5	9	3	4
RR, $q = 4$	yes	4.5	8.5	3.25	4
RR, $q = 2$	yes	5	9	1.5	8

All times in ms. On this small job set SRTF wins every column, but it needs to know the future and can starve long jobs. RR needs no knowledge at all and guarantees every process a turn within $(n-1)q$. FCFS is the simplest. Priority scheduling optimises importance, not averages. A different job set can change the order in the middle of the table, but SRTF always gives the minimum average waiting time when switches are free.

7 Race Conditions

In Part 1 the scheduler preempted processes whenever it liked. For independent processes that is harmless. But the threads of one process share their global variables and heap (Week 4), and a preemption, or a second CPU, can then corrupt shared data.

7.1 `counter++` is not one step

Consider a global variable `counter` that two threads increment. In C this is one statement, but the CPU executes three instructions. This is the real output of `gcc -O2 -S` for our program `race.c` on x86-64:

```
movq  counter(%rip), %rax     # 1. LOAD  counter from memory into register rax
addq  $1, %rax                # 2. ADD   1 inside the register
movq  %rax, counter(%rip)     # 3. STORE the register back to memory
```

The variable `counter` lives in shared memory, but the register `rax` is private to each thread: on a context switch its value is saved in the thread's control block and restored later. A switch can happen between any two of the three instructions, and on a multiprocessor the other thread can run at the same time on another CPU.

One unlucky interleaving

`counter` starts at 5. Threads A and B each execute `counter++` once, so the correct result is 7.

Step	Thread A	Thread B	A's rax	B's rax	counter
1	load		5	–	5
2		load	5	5	5
3	add 1		6	5	5
4		add 1	6	6	5
5	store		6	6	6
6		store	6	6	6 (should be 7)

B's store overwrites A's: one increment is lost. Other orders (for example A's three steps, then B's three steps) give the correct 7. The result depends on the timing.

7.2 Definition

Race condition

A **race condition** occurs when several threads or processes access the same data concurrently, at least one of them writes it, and the outcome depends on the particular order in which the accesses take place.

Everyday examples: two ATMs withdraw money from the same account at the same moment and both read the old balance; two booking servers both see "one seat left" and both sell it. Inside the kernel: if two processes call `fork()` at the same time and the kernel's variable `next_available_pid` is not protected, both children could receive the same process ID. Race conditions are particularly nasty because they are *timing bugs*: a program can pass a thousand tests and fail on the next run, or only on a faster machine. They must be prevented by design.

8 The Critical-Section Problem

A **critical section** is a segment of code in which a thread accesses shared data (a variable, a list, a file). The **critical-section problem** is to design a protocol that the threads can use so that no two of them execute their critical sections at the same time. Each thread follows the structure:

```
while (true) {
    /* entry section:     ask for permission         */
        /* critical section: use the shared data     */
    /* exit section:      announce that we are done  */
    /* remainder section: everything else            */
}
```

A correct solution must satisfy three requirements:

1. **Mutual exclusion.** If a thread is executing in its critical section, no other thread can be executing in its critical section.
2. **Progress.** If no thread is in its critical section and some threads want to enter, only those threads take part in deciding who enters next, and the decision cannot be postponed forever. (Threads in their remainder section have no veto.)
3. **Bounded waiting.** After a thread has requested entry, there is a limit on the number of times other threads may enter their critical sections before it. In other words: no starvation.

We assume that every thread runs at some non-zero speed, but we make *no* assumption about the relative speeds of the threads or the number of CPUs.

Each requirement rules out a "cheap" solution: *never let anyone in* satisfies mutual exclusion but not progress; *strict alternation* (A, B, A, B, ...) violates progress when B does not want to enter; a lock that always prefers A can violate bounded waiting for B.

8.1 Why not just disable interrupts?

On a single CPU, a running thread loses the CPU only through an interrupt (or by blocking voluntarily). So a thread could disable interrupts at the start of its critical section and enable them again at the end; no context switch, no race. Early UNIX kernels did exactly this. But:

- On a **multiprocessor** the other CPUs keep running and can still access the shared data. Disabling interrupts on *all* CPUs requires sending a message to each of them, which is slow and stalls the whole machine.
- Disabling interrupts is a **privileged instruction** (Week 2): user programs cannot do it, and must not be allowed to, since a program that never re-enables them would own the machine.
- During a long critical section the clock and the I/O devices are not served.

Linux still disables interrupts on the *local* CPU for a few instructions inside the kernel, but always together with a lock that works across CPUs.

9 Peterson's Solution

Peterson's solution (Gary Peterson, 1981) is a classic software-only solution for two threads, P_0 and P_1. It uses two shared variables:

```
int  turn;       /* whose turn is it to wait?          */
bool flag[2];    /* flag[i] == true: Pi wants to enter */

/* code for thread i;  j = 1 - i is the other thread */
while (true) {
    flag[i] = true;              /* I want to enter ...     */
    turn = j;                    /* ... but you may go first */
    while (flag[j] && turn == j)
        ;                        /* busy wait */
    /* critical section */
    flag[i] = false;             /* I am done */
    /* remainder section */
}
```

Why it works.

- *Mutual exclusion.* If both threads were in the critical section, both flags would be true. Then each thread must have left its waiting loop because `turn` was equal to its own number, but `turn` holds only one value. Concretely: the thread that wrote `turn` *last* gave the turn away and must wait.
- *Progress.* A thread waits only while the other one wants to enter *and* has the turn; if the other one is in its remainder section, `flag[j]` is false and the thread enters at once.
- *Bounded waiting.* When P_j leaves, it sets `flag[j] = false`. If it immediately wants to enter again, it sets `turn = i` and has to wait. So P_i waits for at most one entry of P_j.

A caveat. Peterson's solution is shown for the *idea*. On modern processors it is not guaranteed to work as written: to run faster, CPUs and compilers may *reorder* independent memory accesses (for example, the read of `flag[j]` may effectively happen before the write of `flag[i]`). Correct versions need *memory barriers*. In practice we use the hardware support of the next section.

10 Hardware Support: Atomic Instructions

Modern CPUs provide special instructions that read and modify a memory word as one **atomic** (indivisible) operation: no other CPU can access that word in between. The simplest to explain is `test_and_set`. Its *effect* can be written in C, but remember that the hardware executes it as a single step:

```
bool test_and_set(bool *target) {   /* executed ATOMICALLY */
    bool old = *target;
    *target = true;
    return old;
}

bool lock = false;                  /* shared; false = free */

while (true) {
    while (test_and_set(&lock))
```

```
        ;                           /* spin: it was already taken */
    /* critical section */
    lock = false;                   /* release */
    /* remainder section */
}
```

If `test_and_set` returns `false`, the lock was free, and the same atomic step has already marked it as taken: this thread owns the lock. If it returns `true`, another thread holds the lock and we try again. Two threads can never both see `false`, because their atomic operations are executed one after the other. This simple lock gives mutual exclusion and progress; bounded waiting requires a little more (for example a *ticket lock*, which serves waiting threads in arrival order).

Real instructions: on x86, `xchg` (atomic exchange) and `lock cmpxchg` (**compare-and-swap**, CAS: "if the word still has the value I expect, replace it with a new value"); on ARM, the pair `ldxr`/`stxr`. The C11 standard makes them available portably, for example as the functions `atomic_flag_test_and_set()` and `atomic_compare_exchange_strong()`.

11 Mutex Locks

Application programmers rarely use atomic instructions directly. Operating systems and libraries wrap them in a **mutex lock** (from *mutual exclusion*) with two operations:

```
acquire(&m);        /* wait until m is free, then take it */
    /* critical section */
release(&m);        /* give it back */
```

Calls to `acquire` and `release` must themselves be atomic, which is what the hardware instructions are for. In POSIX threads the operations are called `pthread_mutex_lock()` and `pthread_mutex_unlock()`.

There are two ways to wait for a lock:

- **Busy waiting (spinlock).** The waiting thread loops, testing the lock again and again. This wastes CPU time, but no context switch is needed. Spinlocks are a good choice on multiprocessors when critical sections are very short: as a rule of thumb, if the lock will be held for less than the time of two context switches. On a single CPU spinning is pointless, since the lock holder cannot run while we spin.
- **Sleeping.** The waiting thread is blocked (put into a waiting queue) and the CPU runs another thread; `release` wakes it up. This is better for long critical sections. Linux implements pthread mutexes with *futexes* (fast user-space mutexes): taking a free lock is a single atomic instruction with no system call; only when the lock is busy does the thread ask the kernel to put it to sleep.

12 Semaphores

A **semaphore**, introduced by Edsger Dijkstra in 1965, is an integer variable S that, apart from initialisation, is accessed only through two atomic operations:

```
wait(S) {           /* Dijkstra: P, from Dutch "proberen" (to test) */
    while (S <= 0)
        ;           /* busy wait */
```

```
    S--;
}

signal(S) {          /* Dijkstra: V, from Dutch "verhogen" (to increase) */
    S++;
}
```

In POSIX the operations are called `sem_wait()` and `sem_post()`.

Binary and counting semaphores.

- A **binary semaphore** takes only the values 0 and 1. Initialised to 1, it behaves like a mutex lock: `wait` before the critical section, `signal` after it.
- A **counting semaphore** can take any non-negative value. Initialised to N, the number of identical resources (say $N = 3$ printers), it lets at most N threads use the resource at once: each thread calls `wait` before using a printer and `signal` afterwards. The fourth thread blocks until a printer is free.

Semaphores for ordering. Semaphores can also enforce an order. Suppose statement S_2 in thread B must run only after statement S_1 in thread A. Use a semaphore `synch` initialised to 0: thread A executes S_1; `signal(synch)`, and thread B executes `wait(synch)`; S_2. Whatever the scheduler does, B cannot pass its `wait` before A has signalled.

Implementation without busy waiting. The definition above spins. Real implementations attach a *waiting queue* to each semaphore: if `wait` finds no free resource, the thread is blocked and put in the queue; `signal` removes one thread from the queue and makes it ready. Some textbook versions let S become negative, in which case $-S$ is the number of waiting threads. The operations themselves are made atomic using the hardware instructions of the previous section, so a little busy waiting remains, but only for the few instructions of `wait` and `signal`, not for the whole critical section.

13 The Bounded-Buffer Problem

A **producer** thread generates items (for example, decoded video frames) and puts them into a buffer with n slots; a **consumer** thread takes them out (and displays them). The producer must wait when the buffer is full; the consumer must wait when it is empty; and the two must never modify the buffer indices at the same time. Three semaphores solve the problem:

Semaphore	Initial value	Meaning
`empty`	n	number of free slots (counting)
`full`	0	number of filled slots (counting)
`mutex`	1	lock for the buffer and its indices (binary)

```
/* producer */
while (true) {
    /* produce an item */
    wait(empty);
    wait(mutex);
    /* add item to buffer */
    signal(mutex);
    signal(full);
}
```

```
/* consumer */
while (true) {
    wait(full);
    wait(mutex);
    /* remove item from buffer */
    signal(mutex);
    signal(empty);
    /* consume the item */
}
```

The roles are symmetric: the producer "consumes" free slots and "produces" full ones; the consumer does the opposite. When the buffer is full, `empty` $= 0$ and the producer blocks in `wait(empty)` until the consumer signals. The order of the two `wait` calls matters, as the next section shows. A real C version with POSIX semaphores is in Section 15.3.

14 A Warning: Deadlock

Careless use of locks can make threads wait for each other forever. Let S and Q be two semaphores initialised to 1:

```
/* thread P0 */
wait(S);
wait(Q);
  ...
signal(S);
signal(Q);
```

```
/* thread P1 */
wait(Q);
wait(S);
  ...
signal(Q);
signal(S);
```

If P0 executes `wait(S)` and is then preempted, P1 can execute `wait(Q)`. Now P0 blocks in `wait(Q)` and P1 blocks in `wait(S)`: each waits for a `signal` that only the other can perform. This is a **deadlock**. The simplest prevention rule is to make all threads acquire locks in the same global order (here: always S before Q).

The bounded buffer has the same trap. If the producer called `wait(mutex)` *before* `wait(empty)` while the buffer is full, it would sleep while holding `mutex`; the consumer could then never enter to remove an item, and both would wait forever. Note the difference from starvation: in a deadlock *nobody* in the group makes progress; in starvation others do. Deadlock handling in full (detection, avoidance, the banker's algorithm) is beyond the scope of this course.

15 Hands-on: Threads, Races and Locks in C

The programs are in the `code/` folder; type `make` to build them all (each is compiled with `gcc -Wall -O2 -pthread`). They use POSIX threads: `pthread_create()` starts a new thread running a given function, and `pthread_join()` waits for it to finish. As in Week 2, you are not expected to write C; read the comments and focus on what happens.

15.1 Example 1: a real race condition

```c
/* volatile: force a real load and store on every increment
 * (otherwise -O2 would add N in one step and hide the race). */
volatile long counter = 0;       /* shared by both threads */

void *worker(void *arg)
{
    for (long i = 0; i < N; i++)
        counter++;               /* load, add 1, store: NOT atomic */
    return NULL;
}
int main(void)
{
    pthread_t a, b;
    double t0 = seconds();
    pthread_create(&a, NULL, worker, NULL);   /* start thread A */
    pthread_create(&b, NULL, worker, NULL);   /* start thread B */
    pthread_join(a, NULL);                    /* wait for both  */
    pthread_join(b, NULL);
    double t1 = seconds();

    printf("counter = %8ld of %ld  (lost %7ld)  %.3f s\n",
           counter, 2 * N, 2 * N - counter, t1 - t0);
    return 0;
}
```

The variable is declared `volatile` so that the compiler really loads and stores it on every iteration; otherwise the optimiser would replace the loop by a single addition and hide the race. Four runs, then two runs pinned to a single CPU with `taskset`, on our build machine (2 CPUs):

```
$ ./race          # run 4 times
counter = 12250428 of 20000000  (lost 7749572)  0.052 s
counter = 12005053 of 20000000  (lost 7994947)  0.100 s
counter = 13797565 of 20000000  (lost 6202435)  0.110 s
counter = 12046532 of 20000000  (lost 7953468)  0.080 s
$ taskset -c 0 ./race   # pinned to ONE CPU
counter = 14324099 of 20000000  (lost 5675901)  0.034 s
counter = 12867601 of 20000000  (lost 7132399)  0.034 s
```

Between 28% and 40% of the increments were lost, and the result was different every time. With two CPUs the threads really run at the same time, and their load–add–store sequences overlap constantly. Perhaps surprisingly, the losses remain large on *one* CPU. The reason: when a thread is preempted between its load and its store, the other thread runs for a whole time slice, millions of increments; when the first thread resumes, it stores its old, stale value and wipes out all of that work in one instruction.

15.2 Example 2: the fix with a mutex

```c
void *worker(void *arg)
{
    for (long i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);     /* acquire: entry section */
        counter++;                     /* critical section       */
```

```
        pthread_mutex_unlock(&lock);   /* release: exit section  */
    }
    return NULL;
}
```

```
$ ./race_fixed    # run 3 times
counter = 20000000 of 20000000  (lost       0)  1.122 s
counter = 20000000 of 20000000  (lost       0)  1.115 s
counter = 20000000 of 20000000  (lost       0)  0.749 s
```

The result is now always correct, but the program is about 10 to 20 times slower (0.75–1.12 s instead of 0.05–0.11 s). It performs 2×10^7 lock/unlock pairs, and the two CPUs keep passing the lock (and the cache line that holds it) back and forth. The cost per locked increment is

$$\frac{0.749\ \text{s}}{2 \times 10^7} \approx 37\ \text{ns} \quad \text{to} \quad \frac{1.122\ \text{s}}{2 \times 10^7} \approx 56\ \text{ns}.$$

Lesson: correctness first, then performance. A better design keeps the critical section rare: each thread counts in a private local variable and adds its total to `counter` once, under the lock. Then only two lock operations are needed instead of twenty million.

15.3 Example 3: producer and consumer with POSIX semaphores

The producer and the start of `main()` (the consumer is the mirror image, exactly as in the pseudocode; full program in `code/prodcons.c`):

```
void *producer(void *arg)
{
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);               /* wait for a free slot   */
        sem_wait(&mutex);               /* enter critical section */
        buffer[in] = item;
        in = (in + 1) % SIZE;
        sem_post(&mutex);               /* leave critical section */
        sem_post(&full);                /* one more filled slot   */
    }
    return NULL;
}
int main(void)
{
    pthread_t p, c;
    sem_init(&empty, 0, SIZE);   /* 0 = shared between threads, not processes */
    sem_init(&full,  0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
```

```
$ ./prodcons
consumed  5   running sum  15
consumed 10   running sum  55
consumed 15   running sum 120
consumed 20   running sum 210
sum of 1..20 = 210 (expected 210)
```

The sum $1 + 2 + \cdots + 20 = 20 \cdot 21/2 = 210$ shows that every item arrived exactly once: none was lost or duplicated, even though the buffer has only 5 slots and the producer is repeatedly blocked when it gets 5 items ahead. The second argument 0 of `sem_init()` means that the semaphore is shared between the threads of one process (a non-zero value would allow sharing between processes through shared memory).

16 Summary

Week 5 in seven sentences

1. FCFS is simple but suffers from the convoy effect: the same three jobs give an average wait of 17 or 3 ms depending only on their order.
2. SJF minimises the average waiting time when all jobs are ready together, SRTF when they arrive over time; both need the next burst length, predicted by $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
3. Priority scheduling can starve low-priority processes; aging raises their priority over time.
4. Round Robin bounds every wait by $(n - 1)q$; a too large q gives FCFS, a too small one wastes the CPU on switching. Linux schedules normal tasks for fairness (CFS, EEVDF since 6.6), weighted by nice values.
5. A race condition arises when concurrent threads access shared data, at least one writes, and the result depends on timing; `counter++` is load–add–store and can lose updates.
6. A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting; atomic hardware instructions are the basis of mutex locks and semaphores.
7. Semaphores solve the bounded-buffer problem (`empty`, `full`, `mutex`), but locks taken in inconsistent orders can deadlock.

17 Glossary

Aging Gradually raising the priority of a process that has waited a long time, to prevent starvation.

Bounded waiting The requirement that a thread waiting to enter its critical section is overtaken only a limited number of times.

Critical section Code in which a thread accesses shared data.

Deadlock A situation in which every thread of a group waits for an event that only another thread of the group can cause.

Mutex lock A lock with `acquire` and `release` operations that provides mutual exclusion.

Mutual exclusion At most one thread executes in its critical section at any time.

Race condition A situation in which the result of concurrent accesses to shared data depends on their timing.

Semaphore An integer accessed only through the atomic operations `wait` (P) and `signal` (V).

Starvation A ready process that is never given the resource it needs.

Time quantum The maximum time a process may run in one turn under Round Robin.

18 Try It Yourself

Races and schedulers on your own Linux machine

In the `code/` folder (lab PCs, WSL, or a virtual machine):

```
$ make                      # builds race, race_fixed, prodcons, cpu_hog
$ ./race; ./race; ./race    # a different wrong answer each time?
$ time ./race_fixed         # correct, but how much slower?
$ python3 sched_check.py    # all Gantt charts of these notes
```

1. How many updates does `race` lose on your machine? Does pinning it to one CPU (`taskset -c 0 ./race`) change the picture?
2. Change `race_fixed.c` so that each thread counts in a local variable and adds the total to `counter` once, under the lock. How fast is it now? Is it still correct?
3. In `prodcons.c`, swap the two `sem_wait()` calls in the producer and set `SIZE` to 1. What happens, and why?

19 Review Questions

Conceptual

1. Prove, with the exchange argument, that shortest-job-first minimises the average waiting time when all jobs arrive together.
2. Why can SJF not be implemented exactly in a CPU scheduler? How does the OS approximate it? What happens to the prediction when $\alpha = 0$ and when $\alpha = 1$?
3. What is starvation? Which algorithms can cause it? How does aging solve it?
4. Explain in your own words what a race condition is. Why does `counter++` give wrong results when two threads execute it concurrently?
5. State the three requirements for a solution to the critical-section problem. Give a "solution" that satisfies mutual exclusion but not progress.
6. Compare a spinlock and a sleeping mutex. When is each one the better choice?
7. What is the difference between a binary and a counting semaphore? Give a use for each.
8. In the bounded-buffer solution, what does each of the semaphores `empty`, `full` and `mutex` count or protect? What goes wrong if the producer takes `mutex` before `empty`?

Numerical

9. Four processes arrive as follows (times in ms): A (0, burst 6), B (1, burst 3), C (2, burst 8), D (3, burst 2). Draw the Gantt chart and compute the average waiting time for (a) FCFS, (b) non-preemptive SJF, (c) SRTF, (d) RR with $q = 3$ (use our convention for simultaneous events).
10. Using exponential averaging with $\alpha = 0.5$ and $\tau_0 = 12$ ms, the first three bursts of a process are 10, 6

and 8 ms. Compute τ_1, τ_2 and τ_3.

11. A context switch takes 0.5 ms and the quantum is 20 ms. What fraction of the CPU time is lost to switching when all processes use their full quantum?

12. `race_fixed` performed 2×10^7 locked increments in 0.749 s, and `race` performed the same number of unlocked increments in 0.052 s. Compute the time per increment in each case and the slow-down factor.

Answers to the numerical questions

9. (a) FCFS: A 0–6, B 6–9, C 9–17, D 17–19; waits $0, 5, 7, 14$; average $26/4 = 6.5$ ms. (b) SJF: A 0–6, D 6–8, B 8–11, C 11–19; waits $0, 7, 9, 3$ (A, B, C, D); average $19/4 = 4.75$ ms. (c) SRTF: A 0–1, B 1–4, D 4–6, A 6–11, C 11–19; waits A $= 11 - 6 = 5$, B $= 0$, C $= 9$, D $= 1$; average $15/4 = 3.75$ ms. (d) RR, $q = 3$: A 0–3, B 3–6, C 6–9, D 9–11, A 11–14, C 14–19; waits A $= 8$, B $= 2$, C $= 9$, D $= 6$; average $25/4 = 6.25$ ms.

10. $\tau_1 = 0.5 \cdot 10 + 0.5 \cdot 12 = 11$ ms; $\tau_2 = 0.5 \cdot 6 + 0.5 \cdot 11 = 8.5$ ms; $\tau_3 = 0.5 \cdot 8 + 0.5 \cdot 8.5 = 8.25$ ms.

11. $s/(q + s) = 0.5/20.5 \approx 0.024$, i.e. about 2.4%.

12. Locked: $0.749/(2 \times 10^7) \approx 37$ ns; unlocked: $0.052/(2 \times 10^7) = 2.6$ ns; slow-down $\approx 37.45/2.6 \approx 14$. (And the unlocked version is wrong.)

20 Looking Ahead: Week 6

- From the CPU to the second big resource: main memory. Address binding; logical versus physical addresses; the memory-management unit (MMU).
- Swapping and contiguous memory allocation; first fit, best fit, worst fit.
- External and internal fragmentation. Paging: pages, frames and page tables; the TLB and the effective access time; segmentation. (Please review powers of two: $2^{10} = 1024$, $2^{12} = 4096$.)

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapter 5 (CPU Scheduling), Chapter 6 (Synchronization Tools), Section 7.1 (bounded buffer); deadlock in Chapter 8.
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapter PDFs:
  - Ch. 7, Scheduling: Introduction: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`
  - Ch. 8, Multi-Level Feedback Queue: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf`
  - Ch. 26, Concurrency: An Introduction: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf`
  - Ch. 28, Locks: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf`
  - Ch. 31, Semaphores: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf`